

Die Transformer-Architektur – Wie funktionieren moderne KI-Sprachmodelle?

Zielgruppe: Schülerinnen und Schüler der gymnasialen Oberstufe

Voraussetzungen: Grundlegendes Verständnis von Funktionen und Vektoren (Mathematik Klasse 10–11)

Ergänzung: Für vertiefende Aspekte siehe *Transformer – Vertiefung für Lehrkräfte und Interessierte*

Inhaltsverzeichnis

1. Was ist ein Transformer?
 2. Der große Überblick: Wie ist ein Transformer aufgebaut?
 3. Schritt 1: Tokenisierung – Text in Zahlen verwandeln
 4. Schritt 2: Embeddings – Bedeutung als Zahlenvektor
 5. Schritt 3: Positional Encoding – Wo steht welches Wort?
 6. Schritt 4: Self-Attention – Der Kern des Transformers
 7. Schritt 5: Multi-Head Attention – Mehrere Blickwinkel gleichzeitig
 8. Schritt 6: Feed-Forward-Netzwerk – Wissen verarbeiten
 9. Schritt 7: Layer Normalization und Residual Connections
 10. Encoder und Decoder – Zwei Rollen im Transformer
 11. Training: Wie lernt ein Transformer?
 12. Bekannte Modelle und ihre Varianten
 13. Zusammenfassung: Die wichtigsten Konzepte auf einen Blick
 14. Weiterführende Fragen und Aufgaben
-

1. Was ist ein Transformer?

Ein **Transformer** ist eine bestimmte Art von künstlichem neuronalem Netz, das 2017 von Forschern bei Google im Aufsatz „*Attention Is All You Need*“ vorgestellt wurde. Seitdem ist die Transformer-Architektur die Grundlage fast aller modernen KI-Sprachmodelle – darunter ChatGPT, Claude, Gemini und viele andere.

Was kann ein Transformer?

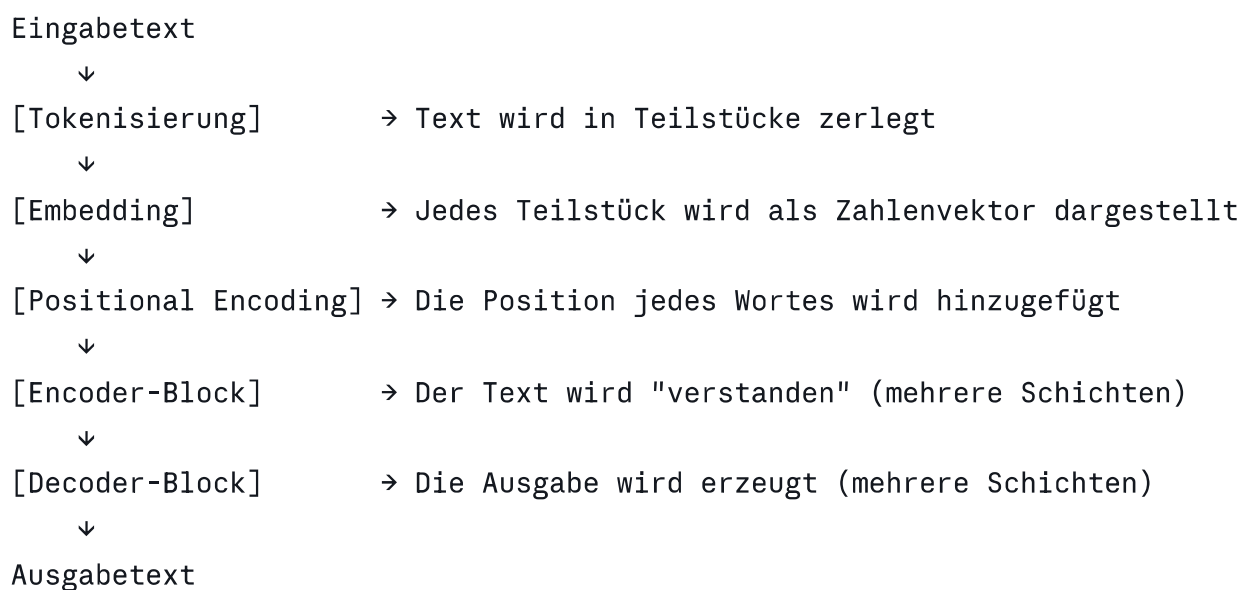
- Texte verstehen und erzeugen (z. B. Fragen beantworten, Texte übersetzen)
- Zusammenhänge zwischen Wörtern erkennen, auch wenn sie weit auseinanderliegen
- Gleichzeitig viele Beziehungen im Text berücksichtigen

Warum war der Transformer eine Revolution?

Vor dem Transformer arbeiteten Sprachmodelle meistens mit sogenannten **rekurrenten neuronalen Netzen (RNNs)**, die Texte Wort für Wort nacheinander (sequenziell) verarbeiteten. Das war langsam und hatte Schwierigkeiten mit langen Texten. Der Transformer verarbeitet alle Wörter **gleichzeitig (parallel)** – das machte ihn viel schneller und leistungsfähiger.

2. Der große Überblick: Wie ist ein Transformer aufgebaut?

Stell dir vor, du willst den Satz „Die Katze, die auf dem Dach saß, schlief.“ ins Englische übersetzen. Ein Transformer arbeitet dabei grob in folgenden Schritten:



Hinweis: Nicht alle Transformer nutzen beide Teile (Encoder und Decoder). Manche Modelle wie GPT nutzen nur den Decoder-Teil, andere wie BERT nur den Encoder-Teil. Mehr dazu in [Abschnitt 10](#).

3. Schritt 1: Tokenisierung – Text in Zahlen verwandeln

Computer können keinen Text direkt verarbeiten – sie arbeiten nur mit Zahlen. Deshalb wird der Eingabetext zuerst in **Tokens** (Teilstücke) zerlegt.

Was ist ein Token?

Ein Token ist nicht immer ein ganzes Wort. Es kann sein:

- Ein ganzes Wort: **Hund** → Token
- Ein Wortteil: **Unglück** → **Un** + **glück** (zwei Tokens)
- Ein Satzzeichen: **.** → Token

Beispiel:

```
Eingabe: "Transformer sind faszinierend!"  
Tokens: ["Trans", "former", " sind", " fasz", "in", "ierend", "!"]  
Zahlen: [4312,      892,      531,      7841,   29,   1047,    54]
```

(Die Zahlen sind erfunden – echte Tokenizer verwenden eigene Wörterbücher)

Jeder Token bekommt eine eindeutige **ID** (eine Ganzzahl), die in einer riesigen Tabelle (dem **Vokabular**) nachgeschlagen wird. Moderne Sprachmodelle haben Vokabulare mit 30.000 bis über 100.000 Einträgen.

4. Schritt 2: Embeddings - Bedeutung als Zahlenvektor

Eine bloße ID-Nummer trägt noch keine inhaltliche Information. Deshalb wird jeder Token in einen **Embedding-Vektor** umgewandelt – eine Liste von Zahlen (z. B. 512 oder 1024 Zahlen), die die Bedeutung des Tokens kodiert.

Was steckt in einem Embedding?

Das Besondere: Ähnliche Bedeutungen führen zu ähnlichen Vektoren. Schau dir folgendes vereinfachtes Beispiel an (nur 2 Dimensionen statt hunderten):

```
"König"   → [0.9, 0.8]  
"Königin" → [0.9, 0.2]  
"Mann"    → [0.2, 0.8]  
"Frau"    → [0.2, 0.2]
```

Ein bekanntes Phänomen: $\text{König} - \text{Mann} + \text{Frau} \approx \text{Königin}$ (vektoriell)

Das zeigt, dass die Embeddings **semantische Beziehungen** zwischen Wörtern erfassen können. Diese Vektoren werden während des Trainings automatisch gelernt.

Mathematisch: Das Embedding ist eine Matrix-Multiplikation: Jede Token-ID wird als One-Hot-Vektor kodiert und mit der Embedding-Matrix multipliziert – das Ergebnis ist der Embedding-Vektor.

5. Schritt 3: Positional Encoding – Wo steht welches Wort?

Da der Transformer alle Tokens **gleichzeitig** verarbeitet (und nicht nacheinander), verliert er zunächst die Information über die **Reihenfolge** der Wörter. Doch die Reihenfolge ist wichtig:

- „Der Hund beißt den Mann.“ $\neq$ „Der Mann beißt den Hund.“

Um die Position jedes Tokens zu kodieren, wird dem Embedding-Vektor ein **Positional Encoding** hinzuaddiert – ein weiterer Vektor, der die Position im Satz beschreibt.

Wie funktioniert das?

Die ursprünglichen Transformer verwenden Sinus- und Kosinus-Funktionen mit verschiedenen Frequenzen:

$$\begin{aligned} \text{PE}(\text{pos}, i) &= \sin(\text{pos} / 10000^{(2i/d)}) && \text{für gerade } i \\ \text{PE}(\text{pos}, i) &= \cos(\text{pos} / 10000^{(2i/d)}) && \text{für ungerade } i \end{aligned}$$

- pos = Position des Tokens im Satz (0, 1, 2, ...)
- i = Dimension im Vektor
- d = Gesamtlänge des Vektors

Anschaulich: Jede Position im Satz hat eine einzigartige „Fingerabdruck-Kombination“ aus verschiedenen Sinuswellen. Das Modell kann daraus ableiten, welches Token wo im Satz steht.

6. Schritt 4: Self-Attention – Der Kern des Transformers

Self-Attention (dt.: Selbst-Aufmerksamkeit) ist das wichtigste Konzept im Transformer. Es erlaubt dem Modell, für jedes Token zu berechnen, **welche anderen Tokens im Satz relevant sind**.

Ein alltagsnahes Beispiel

Betrachte den Satz: „Die Bank am Fluss ist schön.“

Das Wort „Bank“ ist mehrdeutig – es könnte ein Geldinstitut oder eine Sitzgelegenheit sein. Self-Attention hilft dem Modell zu erkennen, dass „Fluss“ und „schön“ relevante Kontext-Tokens sind, die auf die Bedeutung „Sitzbank“ hinweisen.

Query, Key und Value – das Herzstück der Attention

Für jedes Token werden drei Vektoren berechnet:

Vektor	Bedeutung	Analogie
Query (Q)	„Wonach suche ich?“	Eine Suchanfrage
Key (K)	„Was biete ich an?“	Ein Schlagwort/Label
Value (V)	„Was ist mein Inhalt?“	Der eigentliche Informationsgehalt

Ablauf der Self-Attention (vereinfacht):

Für jedes Token t :

1. Berechne Q_t , K_t , V_t durch lineare Transformation
2. Berechne Ähnlichkeit: $\text{score}(t, j) = Q_t \cdot K_j$ (Skalarprodukt)
3. Skaliere: $\text{score} / \sqrt{\text{Dimension von } K}$
4. Normalisiere mit Softmax $\rightarrow$ Attention-Gewichte (Summe = 1)
5. Gewichtete Summe der Values: $\text{output}_t = \sum (\text{Attention-Gewicht}_j \times V_j)$

Beispiel mit Zahlen (stark vereinfacht, 2 Tokens):

Token 1: "Bank" $\rightarrow Q_1 = [1, 0]$, $K_1 = [1, 0]$, $V_1 = [0.8, 0.1]$

Token 2: "Fluss" $\rightarrow Q_2 = [0, 1]$, $K_2 = [0, 1]$, $V_2 = [0.1, 0.9]$

$\text{Score}(\text{"Bank"} \rightarrow \text{"Fluss"}) = Q_1 \cdot K_2 = [1, 0] \cdot [0, 1] = 0$

$\text{Score}(\text{"Bank"} \rightarrow \text{"Bank"}) = Q_1 \cdot K_1 = [1, 0] \cdot [1, 0] = 1$

Nach Softmax: $\text{Attention}(\text{"Bank"}) = [0.73, 0.27]$ $\rightarrow$ Bank schaut hauptsächlich auf sich

Das Ergebnis: Jedes Token erhält einen neuen Vektor, der **kontextbewusst** ist – er enthält Information aus dem gesamten Satz, gewichtet nach Relevanz.

7. Schritt 5: Multi-Head Attention – Mehrere Blickwinkel gleichzeitig

Ein einziger Attention-Mechanismus kann immer nur **eine Art von Beziehung** auf einmal erfassen. Deshalb verwendet der Transformer mehrere Attention-Köpfe (**Multi-Head Attention**) parallel.

Analogie: Ein Text mit verschiedenen Brillen lesen

Stell dir vor, du liest einen Satz mit mehreren unterschiedlichen Brillen:

- **Brille 1** achtet auf grammatikalische Beziehungen (Subjekt-Verb)
- **Brille 2** achtet auf semantische Ähnlichkeit (gleiche Bedeutungsfelder)
- **Brille 3** achtet auf Koreferenz (wer ist „er“ oder „sie“?)
- **Brille 4-8** achten auf weitere Muster

Jeder „Kopf“ (Head) lernt während des Trainings, auf andere Muster zu achten.

Ablauf

Input-Vektor

$\downarrow$ (aufgeteilt in h Teile)

[Head 1] [Head 2] [Head 3] ... [Head h]

$\downarrow$

$\downarrow$

$\downarrow$

$\downarrow$

Self-Attention jeweils separat

↓

Ausgaben zusammengeführt (Konkatenation)

↓

Lineare Projektion → Ausgabe

Typische Transformers haben **8 bis 96 Attention-Köpfe** (je nach Modellgröße).

8. Schritt 6: Feed-Forward-Netzwerk – Wissen verarbeiten

Nach der Multi-Head Attention durchläuft jeder Token-Vektor ein **Feed-Forward-Netzwerk (FFN)** – ein klassisches neuronales Netz mit zwei linearen Schichten und einer Aktivierungsfunktion dazwischen.

Aufbau

Input-Vektor (Dimension d)

↓

Lineare Schicht ($d \rightarrow 4d$) ← Vergrößerung

↓

Aktivierungsfunktion (ReLU oder GELU)

↓

Lineare Schicht ($4d \rightarrow d$) ← Rückverkleinerung

↓

Output-Vektor (Dimension d)

Wozu braucht man das?

Die Attention-Schicht ist gut darin, **Beziehungen** zwischen Tokens zu erkennen. Das Feed-Forward-Netz ist dafür zuständig, das gesammelte Kontextwissen in **konkrete Repräsentationen** umzuwandeln – quasi: „Was bedeutet es für dieses Token, dass es in diesem Kontext steht?“

Interessant: In großen Sprachmodellen wie GPT-4 enthalten die Feed-Forward-Schichten den Großteil der gespeicherten Fakten und Assoziationen – quasi das „Gedächtnis“ des Modells.

9. Schritt 7: Layer Normalization und Residual Connections

Ohne weitere Maßnahmen würde ein tiefes neuronales Netz mit vielen Schichten kaum trainierbar sein – die Fehler würden sich beim Rückwärtsdurchlauf (Backpropagation) verflüchtigen oder explodieren. Zwei Techniken helfen dabei:

Residual Connections (Übersprung-Verbindungen)

Die Eingabe einer Schicht wird **direkt zur Ausgabe addiert**:

```
output = LayerNorm(x + Sublayer(x))
```

Das bedeutet: Jede Schicht muss nur lernen, wie sie den Eingabevektor **verbessern** soll – nicht komplett neu berechnen. Das stabilisiert das Training erheblich.

Analogie: Stell dir vor, du machst Hausaufgaben und eine Lehrkraft gibt dir Korrekturen. Statt die Aufgabe komplett neu zu schreiben, fügst du nur die Korrekturen ein.

Layer Normalization

Die Vektoren werden nach jeder Teiloperation **normalisiert** – das heißt, Mittelwert und Varianz werden auf feste Werte gesetzt. Das verhindert, dass einzelne Werte extrem groß oder klein werden.

10. Encoder und Decoder – Zwei Rollen im Transformer

Der ursprüngliche Transformer besteht aus zwei Hauptteilen:

Der Encoder

- **Aufgabe:** Den Eingabetext vollständig verstehen
- **Benutzt:** Self-Attention (jedes Token kann alle anderen sehen)
- **Typische Anwendung:** Textklassifikation, Sentimentanalyse
- **Bekanntes Modell:** BERT (von Google)

```
Eingabetext → [Encoder-Block] × N → Kontextuelle Repräsentation
```

Der Decoder

- **Aufgabe:** Den Ausgabertext erzeugen (Token für Token)
- **Besonderheit:** Darf beim Generieren nur bereits erzeugte Tokens sehen (nicht in die Zukunft schauen!) → **Masked Self-Attention**
- **Außerdem:** Cross-Attention – der Decoder schaut sich die Encoder-Ausgabe an
- **Bekanntes Modell:** GPT (von OpenAI)

```
[Encoder-Ausgabe] + bisherige Ausgabe → [Decoder-Block] × N → Nächstes Token
```

Übersicht der Modelltypen

Typ	Beispiele	Stärke
Nur Encoder	BERT, RoBERTa	Text verstehen, klassifizieren
Nur Decoder	GPT-2, GPT-4, Claude	Text generieren
Encoder + Decoder	T5, BART	Übersetzen, Zusammenfassen

11. Training: Wie lernt ein Transformer?

Vortraining (Pre-Training)

Transformer werden auf riesigen Textmengen (z. B. das gesamte Internet, Bücher, Wikipedia) vortrainingiert. Dabei lernen sie zwei typische Aufgaben:

1. Next Token Prediction (Decoder-Modelle): Das Modell sieht einen Text und soll das nächste Wort vorhersagen.

Eingabe: "Der Hund sitzt auf der"

Aufgabe: Was kommt als nächstes? → "Matte" (oder "Straße" oder ...)

2. Masked Language Modeling (Encoder-Modelle, z. B. BERT): Ein zufälliges Wort wird „maskiert“ (versteckt), das Modell soll es erraten.

Eingabe: "Der Hund sitzt auf der [MASK]."

Aufgabe: Was stand an [MASK]? → "Matte"

Wie wird optimiert?

- Die Vorhersage des Modells wird mit der richtigen Antwort verglichen → **Verlustfunktion** (z. B. Cross-Entropy Loss)
- Mit **Backpropagation** werden die Gewichte so angepasst, dass der Fehler kleiner wird
- Dies geschieht mit dem **Adam-Optimierer** (einer modernen Variante des Gradientenabstiegs)
- Das Training dauert bei großen Modellen **Wochen bis Monate** auf Tausenden von GPUs

Fine-Tuning (Feinabstimmung)

Nach dem Vortraining wird das Modell oft für spezifische Aufgaben **feinabgestimmt**, z. B.:

- Beantworten von Fragen

- Folgen von Anweisungen (Instruction Tuning)
 - Sicherere und hilfreichere Antworten (RLHF - Reinforcement Learning from Human Feedback)
-

12. Bekannte Modelle und ihre Varianten

Modell	Entwickler	Typ	Besonderheit
BERT	Google	Encoder	Bidirektionales Sprachverständnis
GPT-2 / GPT-4	OpenAI	Decoder	Textgenerierung, sehr groß
T5	Google	Enc.+Dec.	Alles als Text-zu-Text
Claude	Anthropic	Decoder	Fokus auf Sicherheit und Genauigkeit
Gemini	Google	Decoder	Multimodal (Text + Bilder)
LLaMA	Meta	Decoder	Open-Source-Modell

13. Zusammenfassung: Die wichtigsten Konzepte auf einen Blick

TRANSFORMER - Kurzübersicht

1. TOKENISIERUNG Text → Token-IDs (Zahlen)
 2. EMBEDDING Token-IDs → Bedeutungsvektoren
 3. POS. ENCODING + Positionsinformation
 4. SELF-ATTENTION Jedes Token „schaut“ auf alle anderen
→ Query × Key = Gewichte → gewichtete Value-Summe
 5. MULTI-HEAD Mehrere Attention-Köpfe parallel
 6. FEED-FORWARD Wissen verarbeiten und verdichten
 7. LAYER NORM Stabilisierung durch Normalisierung
 8. RESIDUAL CONN. Übersprung-Verbindungen für stabiles Training
 9. ENCODER Text verstehen
 DECODER Text erzeugen (mit Masked Attention)
 10. TRAINING Next-Token-Prediction / Masked LM
→ Backpropagation + Adam-Optimierer
-

14. Weiterführende Fragen und Aufgaben

Verständnisfragen

1. Warum reicht eine einfache Token-ID nicht aus, und warum braucht man Embeddings?
2. Erkläre in eigenen Worten, was Self-Attention macht.
3. Warum ist Positional Encoding nötig, obwohl Wörter doch in einer bestimmten Reihenfolge im Text stehen?
4. Was ist der Unterschied zwischen einem Encoder-Modell (z. B. BERT) und einem Decoder-Modell (z. B. GPT)?

Vertiefende Aufgaben

5. **Embedding-Experiment:** Suche online nach Word2Vec-Demos und probiere selbst aus, ob $\text{König} - \text{Mann} + \text{Frau} \approx \text{Königin}$ gilt.
6. **Attention visualisieren:** Schau dir die Website [BertViz](#) an. Was fällt dir beim Betrachten der Attention-Muster auf?
7. **Modelle vergleichen:** Suche heraus, wie viele Parameter (Gewichte) BERT-base, GPT-2 und GPT-4 haben. Was sagt das über die Komplexität der Modelle aus?
8. **Tokenizer ausprobieren:** Gehe auf platform.openai.com/tokenizer und gib verschiedene Texte ein. Was fällt dir bei deutschen Wörtern auf?

Diskussionsfragen

9. Transformer-Modelle lernen aus Texten aus dem Internet. Welche Probleme könnte das mit sich bringen?
10. Warum ist es sinnvoll, dass ein Decoder bei der Textgenerierung nicht „in die Zukunft schauen“ darf?

Dokument erstellt für den Einsatz im Informatikunterricht der gymnasialen Oberstufe.

Ergänzende Vertiefung: → [transformer_vertiefung_lehrkraefte.md]